PROJECT 1 · BUSINESS INTELLIGENCE TRACK

Retail Customer RFM Segmentation

Complete project walkthrough — from raw transactions to a live dashboard and a retention-budget recommendation.

End-to-end · SQL data prep → RFM scoring → Power BI → published portfolio piece

THE PROJECT IN ONE PAGE

What this project delivers

Problem

A retailer has a FIXED retention budget — which customers should receive it before they churn?

Decision

Target the valuable-but-slipping (At Risk); skip loyal Champions and fully-lapsed customers.

Data

UCI Online Retail II — a UK gift retailer, ~1.07M line items, Dec 2009 – Dec 2011.

Method

CRISP-DM. Rules-based RFM (Recency, Frequency, Monetary) — descriptive, not ML.

Tools

PostgreSQL + DBeaver for the SQL pipeline; Power BI (live-connected) for the dashboard.

Outcome

A segmented customer table + dashboard driving a concrete budget-allocation recommendation.

METHODOLOGY

CRISP-DM — the full arc, complete

1. Business Understanding ✓
2. Data Understanding ✓
3. Data Preparation ✓
4. Modeling (RFM) ✓
5. Evaluation (recommendation) ✓
6. Deployment (Power BI) ✓

Every phase delivered — and validated

1.07M raw rows were cleaned to 776,577, aggregated to 5,852 scored & segmented customers, and surfaced in a live Power BI dashboard ending in a budget recommendation — each stage checked against a known number.

PHASE 1 · BUSINESS UNDERSTANDING

The decision: where to spend a fixed budget

Spending the retention budget evenly is wasteful. Some customers stay regardless; some are already gone; some are on the fence. The budget should chase only the third group.

Why it matters: retaining a customer is far cheaper than acquiring one, and revenue is concentrated in a small share of buyers. The win is a decision — allocating a scarce resource where it changes the outcome.

Champions

Recent, frequent, high-spend

Skip — they'll buy anyway

At-Risk / valuable-but-slipping

Strong past value, fading recency

SPEND HERE — winnable

Lost / low-value

Long gone or tiny spend

Skip — not worth the cost

THE CORE CONCEPT

RFM — three numbers from purchase history

It formalises what a shopkeeper already knows about their regulars — who bought recently, who buys often, who spends big — so it scales to thousands of customers.

Recency	Days since the last purchase	**Reversed — FEWER days is better (recent = engaged)**
Frequency	Count of DISTINCT invoices (orders)	**Higher is better (loyal / habitual buyer)**
Monetary	Total money spent (SUM of qty × price)	**Higher is better (high-value customer)**

Rules-based, not ML: explicit score thresholds (e.g. R≥4 & F≥4 → Champion) — transparent and defensible; you can always explain why a customer is in a segment.

MAKING IT CONCRETE

Worked example — snapshot date 2011-12-10

Customer	Last order	Recency	Orders	Spend	R F M	Segment
Alice	2011-12-05	5 days	12	£2,400	5 5 5	Champion — skip
Bob	**2011-05-24**	**~200 days**	**8**	**£1,800**	**2 4 4**	**At Risk — TARGET**
Carol	2011-12-07	3 days	1	£45	5 1 1	New — nurture

The budget goes to Bob. He was excellent (8 orders, £1,800) but his recency has collapsed to ~200 days — winnable if reached in time. Alice buys anyway; Carol is too new. RFM's job is to surface the Bobs.

PHASE 2 · DATA UNDERSTANDING

The dataset — profiled before any SQL

1,067,371	8	5,942	2011-12-10
raw line-item rows	columns (all text on load)	distinct customers	snapshot date (Recency anchor)

Facts that shaped every later decision

Grain: one row = one product line, not one order → Frequency counts distinct invoices.
Snapshot date: data ends 2011-12-09, so Recency is measured against 2011-12-10 — never “today.”
Customer ID quirk: stored as a float string “13085.0” → forces text staging + a two-step cast.
Coverage limit: 15.4% of revenue sits on rows with no customer — unscorable, documented.

PHASE 2 · DATA QUALITY

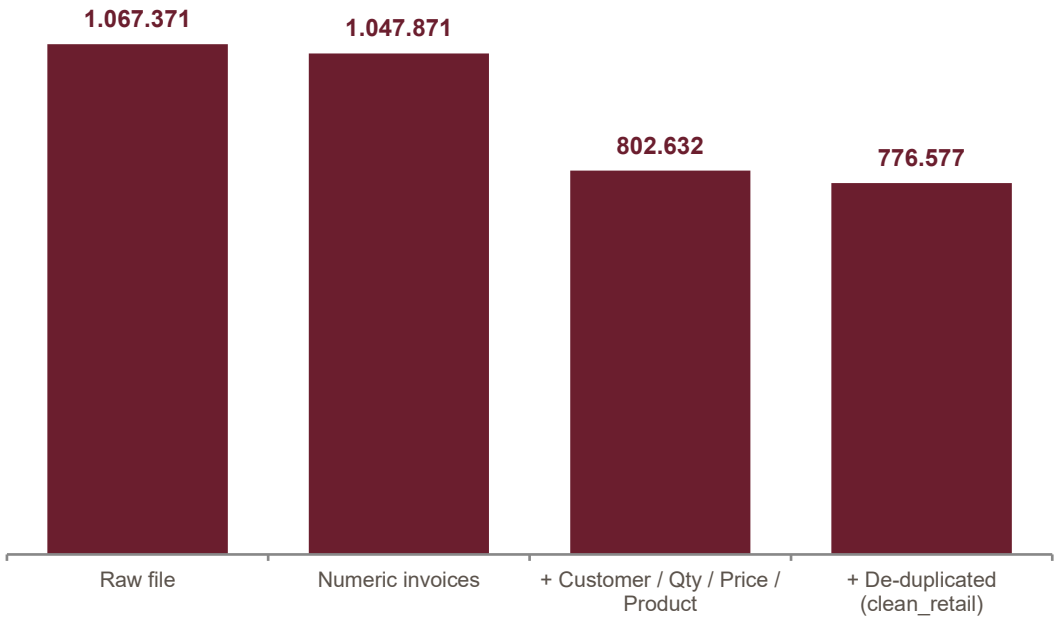
Issues found in the raw data

Issue	Magnitude	Cleaning decision
Missing Customer ID	243,007 (22.8%)	Exclude — RFM is per-customer
Cancellations (invoice 'C')	19,494 (1.8%)	Drop
Adjustment invoices ('A')	6 rows	Drop
Zero / negative price	6,207	Drop — distorts Monetary
Non-product stock codes	6,093 (POST, DOT, M…)	Drop — keep digit-led codes
Exact duplicate rows	34,335 (3.2%)	Drop — double-logging artifacts

Principle: profile first (in Python) to attach real magnitudes to every issue — so “cleaning steps” become concrete, checkable rules with predicted counts.

PHASE 3 · DATA PREPARATION

The cleaning funnel — a validation tool



Why a funnel?

Each filter's row drop was predicted in Python first.

So when the SQL returns the SAME number, the rule is proven correct — not just plausible.

Final: 776,577 rows kept (72.8%), covering 5,852 of 5,942 customers.

PHASE 3 · SQL PIPELINE

Three tables — raw → clean → customer

staging_retail

1,067,371 rows

Raw CSV loaded as-is, every column TEXT. A landing zone that can't fail on messy values.

clean_retail

776,577 rows

Six validated cleaning rules + type casts, built with CREATE TABLE AS. Correctly typed.

customer_rfm

5,852 customers

One row per customer: R/F/M values, 1–5 scores, and a segment label. The dashboard source.

Validate at every hop: each table's row count was checked against a known number before building the next — 1,067,371 → 776,577 → 5,852.

SKILLS USED

The SQL toolkit, by concept

Casting

value::type turns text into numbers/dates. “13085.0” needs ::numeric::integer.

WHERE + regex

Filters rows. ~ matches a regex: ^[0-9]+$ (all digits) vs ^[0-9] (starts with digit).

GROUP BY + aggregates

Collapses rows to one per group. Frequency = COUNT(DISTINCT invoice), not COUNT(*).

Window functions

NTILE(5) OVER(...) ranks customers into quintiles while keeping every row.

CTEs (WITH … AS)

Named steps chained aggregate → score → segment, keeping multi-step SQL readable.

CASE

If/then/else mapping scores to segments; first matching WHEN wins, so order matters.

PHASE 4 · MODELING

Scoring with NTILE, segmenting with CASE

Score → 1–5 quintiles

NTILE(5) splits customers into five equal buckets on each metric — top fifth scores 5.

```
NTILE(5) OVER (ORDER BY recency DESC)
NTILE(5) OVER (ORDER BY frequency ASC)
NTILE(5) OVER (ORDER BY monetary ASC)
```

The Recency flip: sorted DESC so the most-recent buyers score 5 (fewer days = better).

Segment → named groups

A CASE expression maps R and F scores to business segments (first match wins):

- `R≥4 & F≥4` → Champions
- `F≥4` → Loyal
- `R≥4 & F≤3` → Potential Loyalist
- `R≤2 & F≥3` → At Risk
- `R≤2 & F≤2` → Hibernating
- `else` → Others

PHASE 5 · EVALUATION

The result — customers & revenue by segment

Segment	Customers	% Rev	Avg Recency
Champions	~1,470	69%	20 days
Loyal	~870	16%	200 days
Potential Loyalist	~870	5%	27 days
Hibernating	~1,515	3.6%	458 days
At Risk	**~472**	**3.3%**	**385 days**
Others	~655	3.1%	109 days

The concentration insight

Revenue and headcount concentrate DIFFERENTLY.

Champions: ~25% of customers but ~69% of revenue.

Hibernating: the largest group, yet nearly the smallest revenue.

At Risk: the SMALLEST segment by count — yet holds revenue near the 3× larger Hibernating group.

The recommendation

Focus the fixed retention budget on the At-Risk segment.

472	£565K	385
At-Risk customers	lifetime value at stake	avg days since last order

Why them: a proven, high-value buying habit that has lapsed — the highest-ROI win-back. Champions (69% of revenue) are already loyal and need no spend; Hibernating customers warrant only low-cost reactivation. Prioritise outreach by spend, highest-value first.

PHASE 6 · DEPLOYMENT

The Power BI dashboard

KPI header

Total Revenue (£17M), Total Customers (5,852), At-Risk Customers (472) — the state at a glance.

Segment charts

Customers-by-segment and revenue-by-segment side by side — the count-vs-value contrast.

At-Risk call list

A filtered, spend-ranked table of the 472 target customers — a literal retention call list.

Recommendation callout

A highlighted text box stating the decision — the dashboard answers itself in 10 seconds.

Live-connected + DAX

Connected live to PostgreSQL (Import mode). Measures written in DAX:

```
Total Revenue =
  SUM(monetary)

At Risk Customers =
  CALCULATE(
    COUNTROWS(...),
    segment="At Risk")
```

THE META-SKILL

Bugs caught — and how

1

234,245 rows slipped past a filter

Cause: Blank customers loaded as '' not NULL, so IS NOT NULL removed nothing.

Lesson: *The count mismatch (1,036,877 vs 802,632) exposed it instantly.*

2

Columns loaded into the wrong place

Cause: Renamed columns broke name-based CSV mapping → extra columns, ours NULL.

Lesson: *Diagnosis lived in the DATA, not the layout — map by position.*

3

Power BI “unreachable network”

Cause: localhost resolved to IPv6 (::1), which the driver couldn't reach.

Lesson: *Use 127.0.0.1 (explicit IPv4) — read the error's exact wording.*

Overarching discipline: validate every stage against an independently-computed known number.

Key takeaways

- CRISP-DM front-loads understanding — “clean” and “correct” are defined before code.
- RFM is rules-based and explainable; the budget targets the valuable-but-slipping (At Risk).
- The SQL toolkit: staging + casting, WHERE/regex, GROUP BY, NTILE windows, CTEs, and CASE.
- The insight is a mismatch: revenue concentrates in Champions, headcount in Hibernating.
- A BI deliverable ends in a DECISION and an action — not just a chart.
- Validate every step against a known number — the discipline that catches silent bugs.